0.65.3. La fonction defile() — filer sur sa ligne sans arrêter le jeu

#include <defile> : defile(numero, x, y, tuile, sens, vitesse), appelée à chaque image, fait filer une tuile sur sa ligne, d'un bord à l'autre, sans fin.

1. Ce qu'il faut comprendre

Ce qu'elle fait : `defile(…)` fait **filer** une tuile sur sa ligne, sans bloquer : un pas quand c'est l'heure, et au bord elle **repart de l'autre côté**.

La ligne à écrire : `#include <defile>`**.** À ne pas confondre avec `defiler()`, qui fait glisser tout le fond.

Ses arguments : un numéro (0 à 3), le départ, la tuile, le sens (1 : à droite, -1 : à gauche), la vitesse en ms.

Ce qu'elle coûte : environ **300 octets**, plus la police entière.

Essaie : le sens -1.

> **On retrouve defile dans 4 leçons**
>
> 0.65.4, 0.76.13, 0.76.14, 0.90.2

📖 Les mots de cette leçon — si tu débutes

- **accolades { }** — Elles entourent un bloc : les lignes qui vont ensemble (le corps d'une fonction, d'une boucle, d'un « if »). Chaque « { » a son « } ».
- **argument** — Une valeur donnée à une fonction quand on l'appelle, entre ses parenthèses : dans texte(5, 8, "SALUT"), 5, 8 et "SALUT" sont les arguments.
- **bloc** — Un groupe de lignes entre accolades { }, qui s'exécutent ensemble.
- **booléen (vrai / faux)** — Le résultat d'une condition : vrai (1) ou faux (0). « true » veut dire vrai, « false » faux.
- **boucle** — Des instructions qui se répètent : « while » (tant que…) ou « for » (pour un compteur qui va de… à…).
- **boucle du jeu** — « while (true) { image(); … } » : ce qui se répète 60 fois par seconde, sans fin, pendant tout le jeu.
- **commentaire (//)** — Tout ce qui suit « // » jusqu'au bout de la ligne : une phrase pour l'humain, que la console ignore.
- **condition** — Une question dont la réponse est vrai ou faux : « score > 10 », « bouton(A) ». On la teste avec « if » ou « while ».
- **image()** — Attend l'image suivante. La console en montre 60 par seconde : c'est ce qui règle la vitesse d'un jeu.
- **#include** — En haut du programme : « #include <texte> » demande la fonction texte() à la console ; « #include "fichier.cpp" » verse un de tes fichiers à cet endroit.
- **indice** — Le numéro d'une case dans un tableau, entre crochets : cases[3] est la quatrième case (on compte à partir de 0).
- **instruction** — Un ordre donné à la console, terminé par un point-virgule : « texte(1, 1, "OK"); ».
- **main** — « int main() { … } » : là où le programme commence. La console exécute d'abord la première ligne de main.
- **point-virgule (;)** — Il termine une instruction, comme le point termine une phrase.
- **while (tant que)** — Répète un bloc tant qu'une condition est vraie.

🔍 Le programme, ligne par ligne — chaque ligne dite en français

`// ---- #include <defile> : filer sur sa ligne sans arrêter le jeu ----`	Un commentaire : une phrase pour celui qui lit (« ---- #include <defile> : filer sur sa ligne sans arrêter le jeu ---- »). La console l'ignore.
`#include <defile>  // defile() : une tuile qui file, sans bloquer`	Demande à la console « defile » (ce que c'est : une tuile qui file sur sa ligne sans arrêter le jeu). Sans cette ligne, le compilateur refuserait de l'employer. (Le commentaire le dit : « defile() : une tuile qui file, sans bloquer ».)
`#include <ALPHABET>`	Demande à la console « ALPHABET » (ce que c'est : les lettres de la police : ALPHABET[0] est le A). Sans cette ligne, le compilateur refuserait de l'employer.
`int main() {`	Le début du programme : la console commence ici, par la première ligne qui suit.
`while (true) {`	La boucle du jeu : tout ce qui est entre les accolades recommence sans fin, 60 fois par seconde avec image().
`image();`	Attendre l'image suivante : la console en montre 60 par seconde. C'est ce qui règle la vitesse du jeu.

`defile(0, 15, 8, ALPHABET[0], 1, 50);  // vers la droite, 50 ms par pas`	Appel de la fonction defile de la console, avec les arguments 0, 15, 8, ALPHABET[0], 1, 50. Ce qu'elle fait : une lettre qui file sur sa ligne sans fin, sans bloquer : à appeler à chaque image. Au premier appel, elle apparaît en (x, y) ; puis un pas toutes les « vitesse » ms, à gauche (sens -1) ou à droite (1), et au bord elle repart de l'autre côté. numero (0 à 3). (Le commentaire le dit : « vers la droite, 50 ms par pas ».)
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1141 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

```
Gravé dans la cartouche :
  démarrage        49 o  ← la console, à l'allumage  — la mise en route de la console
  defile()        237 o  ← main()  — une fonction fournie par le langage
  main()           33 o  ← démarrage  — ta fonction
  VBlank            8 o  ← la console, à chaque image  — compte les images, 60 fois par seconde
  AttendreImage    28 o  ← image()  — attend l'image suivante
  AttendreVBlank   16 o  ← démarrage  — attend que l'écran soit libre
  AttendreAcces    12 o  ← EffacerCases  — attend de pouvoir écrire une case
  AdresseCase      10 o  ← poser(), effacer()  — trouve l'adresse d'une case de l'écran
  EffacerCases     12 o  ← effacer()  — efface des cases
  CopierTuiles     18 o  ← démarrage  — copie les lettres et les dessins en mémoire vidéo
  EffacerCarte     14 o  ← démarrage  — vide les deux cartes de l'écran
  Tuiles          704 o  ← CopierTuiles  — la police entière
Lettres gravées : toutes (poser() reçoit un numéro calculé)
Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EcrireTexte, EcrireNombre, Diviser, Reste, Multiplier,
DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette,
DonneesTransfert, Notes
Fonctions du langage appelées :
  poser()   × 2    26 o sur place, dans defile()  + AdresseCase (10 o)
  effacer() × 1    15 o sur place, dans defile()  + AdresseCase (10 o), EffacerCases (12 o)
  image()   × 1     3 o sur place, dans main()  + AttendreImage (28 o)
  images()  × 1     2 o sur place, dans defile()
Tes fonctions appelées :
  defile()  × 1    26 o sur place, dans main()
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// ---- #include <defile> : filer sur sa ligne sans arrêter le jeu ----

#include <defile>     // defile() : une tuile qui file, sans bloquer
#include <ALPHABET>
```

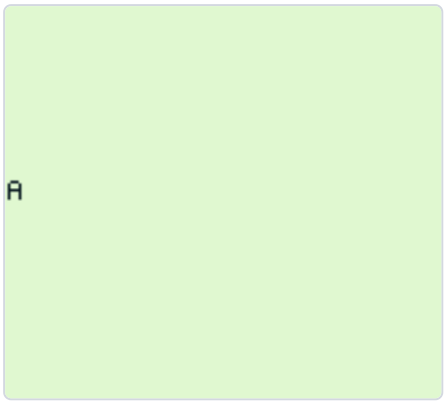
2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
  while (true) {
    image();
    defile(0, 15, 8, ALPHABET[0], 1, 50);   // vers la droite, 50 ms par pas
  }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.

la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60

4. Ce qu'on doit voir

> Un A qui file vers la droite, et revient par la gauche.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ le A atteint le bord droit
- ✓ et repart de la colonne 0

Ensuite : **0.65.4. Une lettre qui avance — en simple** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++